

To split a data set into a train test easily for ML implementation, type:

```
X = list(range(15))
print (X)
y = [x + x^2 for x in X]
print (y)

import sklearn.model_selection as
model_selection
X_train, X_test, y_train, y_test =
model_selection.train_test_split(X, y,
train_size=0.70, test_size=0.30, random_
state=101)
print ("X_train: ", X_train)
print ("y_train: ", y_train)
print("X_test: ", X_test)
print ("y_test: ", y_test)
```

OUTPUT

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11,
12, 13, 14]
[2, 0, 6, 4, 10, 8, 14, 12, 18, 16, 22,
20, 26, 24, 30]
X_train: [5, 0, 14, 4, 8, 9, 7, 6, 1,
11]
y_train: [8, 2, 30, 10, 18, 16, 12,
14, 0, 20]
X_test: [13, 2, 3, 12, 10]
y_test: [24, 6, 4, 26, 22]
```

Matplotlib, Seaborn, Plotly: the essential visualisation libraries

In Python, there are multiple options for visualising data according to need. Because of this variety, it can be really challenging to figure out which one to use when.

Matplotlib is considered as the old grandfather of the Python visualisation packages. It is extremely powerful, but with that power comes some level of complexity. You can typically do anything you need utilising a Matplotlib library, but this is not always so simple to figure out.

Matplotlib originated nearly a decade before Pandas and hence isn't outlined for utilising the Pandas DataFrame. So for visualising data from a Pandas DataFrame, you have to extract each Series and often concatenate them together into the right format. It would be better to have a plotting library that can use the DataFrame labels in a plot.

Seaborn gives an API on Matplotlib that gives rational choices for plot fashion and colour defaults, characterises straightforward high-level capacities for common measurable plot sorts, and coordinates with the usefulness of Pandas.

Plotly is a JSON based plot tool for interactive visualisation. Graphs or plots can be defined by a JSON object using two keys, named 'data' and 'layout'. It can create interactive features easily. Most plots come with hover labels, and legends for groups is another amazing feature of this library.

Tricks to try out!

For a simple scatter plot using Matplotlib library, type:

```
import matplotlib.pyplot as plt
%matplotlib inline
# Draw two sets of points
plt.plot([10,25,30,40,55],
[1,2,3,4,10], 'bo') # blue dots
plt.plot([15,25,50,60,80],
[2,3,4,5,11], 'r*') # red stars
plt.show()
```

The output is given in Figure 2.

Often, in statistical data visualisation, all you want is to plot histograms and joint distributions of variables. We have seen that this is relatively straightforward in Matplotlib. Rather than a histogram, we can get a smooth estimate of the distribution using a kernel density estimation, which Seaborn does with *sns.kdeplot*:

```
import seaborn as sns
sns.set()
import numpy as np
import pandas as pd
data = np.random.multivariate_
normal([0, 1], [[5, 6], [6, 2]],
size=30000)
data = pd.DataFrame(data, columns=['X',
'Y'])

for col in 'XY':
    sns.kdeplot(data[col], shade=True)
```

The output is given in Figure 3.

To use Plotly to draw a scatter plot using the iris data set, type:

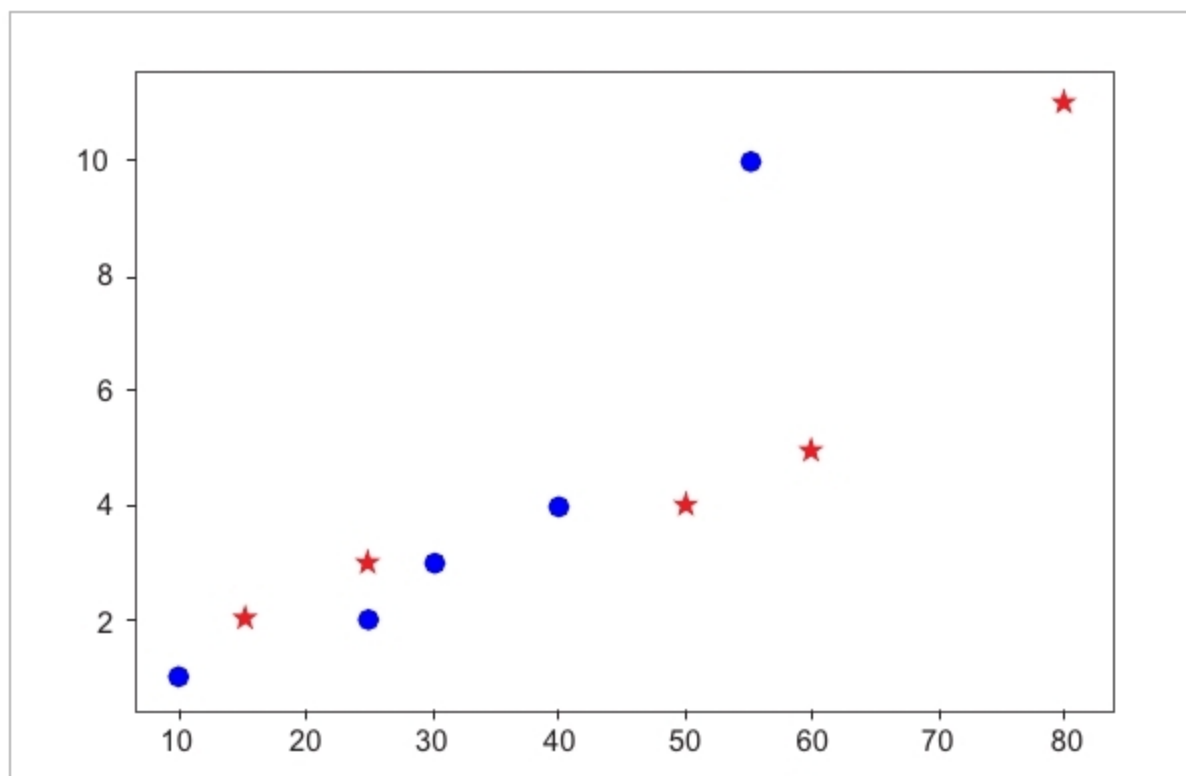


Figure 2: Scatter plot using Matplotlib